
VM Assignment 1

1 Part 1 — Design & Implementation

1.1 Environment Setup

Host: Ubuntu 22.04 LTS on x86

QEMU: v7.0.0

Linux/KVM: v5.15

Disk images: Two Ubuntu 20.04 cloud images: cloud.img (25GB) for the KVM host filesystem, and cloud_inner.img (2GB, sized to fit inside cloud.img) for the nested VM.

1.2 Overall Architecture

The goal of this assignment is to allow a guest VM to pin its vCPU to a specific physical CPU on the KVM host. Achieving this requires modifications to KVM itself, along with a kernel module (hw1_module) running inside the guest that serves as the interface between user space and the hypervisor. To understand why this cannot be handled purely within the guest, the guest kernel has no visibility into these threads. Therefore, any affinity operation must cross the guest/host boundary, which means a VM exit is required.

The same reasoning applies to both directions. When the guest writes a cpumask, the module must trigger a VM exit so that KVM can apply set_cpus_allowed_ptr() to all vCPU kernel threads on the host side. When the guest reads the current cpumask, another VM exit is needed to retrieve the value from the host.

End-to-end flow (write): A user-space program opens /dev/hw1_module and issues a write() with an 8-bit cpumask. The kernel module receives this in device_write() and fires an HVC hypercall, which causes a VM exit and traps into EL2. KVM's hypercall handler dispatches to kvm_assign_affinity(), which iterates over all vCPUs via kvm_for_each_vcpu and calls set_cpus_allowed_ptr() on each vCPU kernel thread with the provided cpumask, pinning the entire VM to the specified physical CPUs.

End-to-end flow (read): A user-space program issues a read() on /dev/hw1_module. The kernel module fires an HVC hypercall via device_read(), trapping into EL2. KVM dispatches to kvm_read_affinity(), which reads current->cpus_ptr — where current is the vCPU kernel thread that handled this VM exit. Since kvm_assign_affinity() applies the same mask to all vCPUs, every vCPU thread carries an identical cpumask, so the returned value is always consistent regardless of which vCPU serves the hypercall.

1.3 KVM-Side Implementation

We begin by tracing the VM exit handling path to understand where to insert our implementation.

```
*/
int handle_exit(struct kvm_vcpu *vcpu, int exception_index)
{
    struct kvm_run *run = vcpu->run;

    exception_index = ARM_EXCEPTION_CODE(exception_index);

    switch (exception_index) {
    case ARM_EXCEPTION_IRQ:
        return 1;
    case ARM_EXCEPTION_EL1_ERROR:
        return 1;
    case ARM_EXCEPTION_TRAP:
        return handle_trap_exceptions(vcpu);
    case ARM_EXCEPTION_HYP_GONE:
        return 1;
    }
```

When a VM exit occurs, KVM enters `handle_exit()`, which dispatches to `handle_trap_exception()`. This function calls `kvm_get_exit_handler()`, which returns a function pointer from `arm_exit_handlers[]` indexed by the ESR exception class. For HVC calls, this resolves to `handle_hvc()`, which ultimately calls `kvm_hvc_call_handler()`

```
int kvm_hvc_call_handler(struct kvm_vcpu *vcpu)
{
    u32 func_id = smccc_get_function(vcpu);
    u64 val[4] = { SMCCC_RET_NOT_SUPPORTED };
    u32 feature;
    gpa_t gpa;

    switch (func_id) {
    case ARM_SMCCC_VERSION_FUNC_ID:
        val[0] = ARM_SMCCC_VERSION_1_1;
        break;
    case ARM_SMCCC_ARCH_FEATURES_FUNC_ID:
        feature = smccc_get_arg1(vcpu);
        switch (feature) {
        case ARM_SMCCC_ARCH_WORKAROUND_1:
            switch (arm64_get_spectre_v2_state()) {
            case SPECTRE_VULNERABLE:
                break;
            case SPECTRE_MITIGATED:
                val[0] = SMCCC_RET_SUCCESS;
                break;
            }
        }
    }
```

```
handle_exit() => handle_trap_exception() => vm_get_exit_handler() =>
arm_exit_handlers[esr_ec] => handle_hvc() => kvm_hvc_call_handler()
```

kvm_hvc_call_handler() selects the target function based on func_id. As seen in arm_hypcalls.h, the function ID is read from the vCPU's register x0, which is also where the guest must place it before issuing the hypercall.

```
static inline u32 smccc_get_function(struct kvm_vcpu *vcpu)
{
    return vcpu_get_reg(vcpu, 0);    ● Use of undeclared identifier 'vcpu'
```

With this call chain understood, we make the following changes. **Define hypercall func_id** in include/linux/arm-smccc.h:

```
#define ARM_SMCCC_ASSIGN_AFFINITY \
    ARM_SMCCC_CALL_VAL(ARM_SMCCC_FAST_CALL, ARM_SMCCC_SMC_64, \
        ARM_SMCCC_OWNER_STANDARD, 0x54)
#define ARM_SMCCC_READ_AFFINITY \
    ARM_SMCCC_CALL_VAL(ARM_SMCCC_FAST_CALL, ARM_SMCCC_SMC_64, \
        ARM_SMCCC_OWNER_STANDARD, 0x55)
```

Register handlers in arch/arm64/kvm/hypcalls.c:

```
case ARM_SMCCC_ASSIGN_AFFINITY:
    return kvm_assign_affinity(vcpu);
case ARM_SMCCC_READ_AFFINITY:
    return kvm_read_affinity(vcpu);
```

Implement the handlers in a new file arch/arm64/kvm/affinity.c, also added to arch/arm64/kvm/Makefile:

```
int kvm_read_affinity(struct kvm_vcpu *vcpu)
{
    u32 val;
    const cpumask_t *mask = current->cpus_ptr;
    bitmap_to_arr32(&val, cpumask_bits(mask), 32);
    printk("kvm_read_affinity: current->pid=%d affinity_mask=%*pbl val=%u\n",
        vcpu->pid, cpumask_pr_args(mask), val);
    unsigned long i;
    struct kvm_vcpu *v;
    kvm_for_each_vcpu (i, v, vcpu->kvm) {
        int pid = pid_vnr(v->pid);
        printk("kvm_read_affinity: vcpu->pid=%d affinity_mask=%*pbl
val=%u\n",
            pid, cpumask_pr_args(mask), val);
    }

    smccc_set_retval(vcpu, val, 0, 0, 0);
    return 1;
}

int kvm_assign_affinity(struct kvm_vcpu *vcpu)
{

```

```

    u32 val = smccc_get_arg1(vcpu);
    cpumask_t mask;
    cpumask_clear(&mask);
    bitmap_from_arr32(cpumask_bits(&mask), &val, 32);

    /* NOTE: By the ordering of spec, it should apply affinity to all vcpu
not just current(pthread)
    * Old approach: only pins the vCPU thread handling this HVC.
    * If the guest process migrates to another vCPU between ASSIGN and READ,
    * the READ will see a different current->cpus_ptr.
    set_cpus_allowed_ptr(current, &mask);
    *
    */

    struct kvm_vcpu *v;
    unsigned long i;
    kvm_for_each_vcpu (i, v, vcpu->kvm) {
        struct task_struct *t;
        rcu_read_lock();
        t = pid_task(rcu_dereference(v->pid), PIDTYPE_PID);
        if (t)
            get_task_struct(t);
        rcu_read_unlock();
        if (t) {
            set_cpus_allowed_ptr(t, &mask);
            put_task_struct(t);
            int pid = pid_vnr(v->pid);
            printk("kvm_assign_affinity: vcpu->pid=%d,
affinity_mask=%*pbl val=%u\n",
                    pid, cpumask_pr_args(&mask), val);
        }
    }

    smccc_set_retval(vcpu, val, 0, 0, 0);
    return 1;
}

```

`kvm_assign_affinity()` applies the cpumask to all vCPU kernel threads via `kvm_for_each_vcpu`, rather than only the thread currently handling the hypercall. This ensures that `kvm_read_affinity()` — which reads `current->cpus_ptr` from whichever vCPU thread serves the READ hypercall — always returns a consistent value, even if the guest process migrates across vCPUs between the two calls.

—

1.4 Guest Kernel Module

`hw1_module.c` implements the bridge between guest user space and the KVM hypercall interface. On `write()`, the module reads an 8-bit cpumask from user space and issues `ARM_SMCCC_ASSIGN_AFFINITY` via an inline HVC instruction, passing the mask in `x1` as required by the SMCCC calling convention. On `read()`, it issues `ARM_SMCCC_READ_AFFINITY` and copies the returned value from `x0` back to user space.

```

static ssize_t hw1_module_read(struct file *file, char __user *buffer,
                               size_t count, loff_t *offset) {
    pr_info("hw1_module: Do the Read Affinity Operations\n");
    uint8_t cpumask;
    register uint64_t x0 asm("x0") = ARM_SMCCC_READ_AFFINITY; // func_id
    asm volatile("hvc #0\n" : "+r"(x0) :: "memory");
    uint8_t result = (uint8_t)x0;
    if (copy_to_user(buffer, &result, 1))
        return -EFAULT;
    return count;
}

static ssize_t hw1_module_write(struct file *file, const char __user *buffer,
                                size_t count, loff_t *offset) {
    pr_info("hw1_module: Do the Assign Affinity Operations\n");
    uint8_t cpumask;
    if (copy_from_user(&cpumask, buffer, 1))
        return -EFAULT;
    register uint64_t x0 asm("x0") = ARM_SMCCC_ASSIGN_AFFINITY; // func_id
    register uint64_t x1 asm("x1") = cpumask;
    asm volatile("hvc #0\n" : "+r"(x0) : "r"(x1) : "memory");
    return count;
}

```

1.5 How to Build and Run

1.5.1 Modified Files

File	Change
include/linux/arm-smccc.h	Define ARM_SMCCC_ASSIGN_AFFINITY and ARM_SMCCC_READ_AFFINITY
arch/arm64/kvm/hypercalls.c	Add dispatch cases for the two new function IDs
arch/arm64/kvm/affinity.c	New file implementing kvm_assign_affinity() and kvm_read_affinity()
arch/arm64/kvm/Makefile	Add affinity.o to the build

1.5.2 Apply patch and rebuild kernel, kernel module

```

cd linux/
git apply b12902078_hw1_kernel.patch
make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- -j4
make KDIR=/path/to/linux ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu-

```

1.5.3 Boot KVM host and Transfer hw1_module.ko to KVM host

```

./run-kvm.sh -k arch/arm64/boot/Image -i cloud.img
scp -P 2222 hw1_module.ko root@localhost:/root
scp -P 2222 /path/to/Image root@localhost:/root
scp -P 2222 /path/to/cloud_inner.img root@localhost:/root
scp -P 2222 /path/to/test.c root@localhost:/root

```

1.5.4 Move test, hw1_module.ko into cloud_inner.img(In KVM host)

```
gcc -static -o test test.c
sudo mount -o loop cloud_inner.img /mnt
sudo cp test /mnt/root/test
sudo cp hw1_module.ko /mnt/root/hw1_module.ko
sudo umount /mnt
```

1.5.5 Run the Guest VM

```
./run-guest.sh -k Image -i cloud_inner.img
```

1.5.6 Install Module and Run the test

```
insmod hw1_module.ko
./test
```

2 Part 2 — Experiment & Evaluation

2.1 Experimental Goal

The goal of this experiment is to verify that the affinity mechanism is correctly implemented on the KVM host — specifically, that writing a cpumask from inside the guest VM causes the corresponding vCPU kernel threads on the KVM host to be pinned to the specified physical CPU(s).

2.2 Setup

The KVM host is booted with 6 vCPUs via QEMU (-smp 6). The nested VM runs with 2 vCPU. The kernel module hw1_module.ko is compiled and installed inside the nested VM via insmod. We use the following tools on the KVM host to observe vCPU thread scheduling:

- `ps -eLo pid,psr,comm` — shows which physical CPU each thread is currently running on (affinity)
- `taskset -p <pid>` — shows the configured Cpus_allowed mask of a thread (affinity)
- `journalctl -k -n 20` confirms that the hypercall reached EL2 and the correct cpumask was received

2.3 Procedure

2.3.1 Install Module and Run the test

```
insmod hw1_module.ko
./test
```

2.3.2 Identify the qemu vcpu

```
ps -eLo pid,psr,comm | grep qemu
htop
```

2.3.3 Record the baseline affinity before pinning

```
taskset -p <vcpu thread_pid>
cat /proc/<vcpu thread_pid>/status | grep Cpus_allowed
```

2.3.4 Run the test and while-loop on Guest VM

```
./test
while true; do ;; done &
```

2.3.5 Confirm the hypercall has been performed

```
journalctl -k -n 20
```

2.3.6 Verify the affinity do apply on KVM host's CPUS

```
taskset -p <vCPU_thread_pid>
cat /proc/<vCPU_thread_pid>/status | grep Cpus_allowed
```

2.4 Results

2.4.1 Identify the vcpu

```
root@ubuntu:~# ps -eLo pid,psr,comm | grep qemu

    988      3  qemu-system-aar
    988      0  qemu-system-aar
```

```
1  [||||||||||||||||||||||||||||| 62.4%] 4  [|||| 5.7%]
2  [||||||| 11.9%] 5  [||| 1.9%]
3  [| 0.6%] 6  [||||||||||||||||||||||||||||| 67.9%]
Mem[||||||||||||||||||||||||||||| 833M/1.93G]
Swp[|||| 0K/0K]
Tasks: 34, 49 thr; 2 running
Load average: 1.62 1.46 0.82
Uptime: 00:08:35
```

PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command
988	root	20	0	4212M	587M	14660	S	139.29.6	11:19.11	qemu-system-aarch64 -nographic -machine virt -m 512 -cpu	
991	root	20	0	4212M	587M	14660	R	77.1 29.6	5:32.37	qemu-system-aarch64 -nographic -machine virt -m 512 -cpu	
992	root	20	0	4212M	587M	14660	S	62.6 29.6	5:31.06	qemu-system-aarch64 -nographic -machine virt -m 512 -cpu	

- Since the Guest VM has two vcpus(pthread), we can know the PID from the http.

2.4.2 Record the baseline affinity before pinning

```
root@ubuntu:~# taskset -p 991
pid 991's current affinity mask: 3f
root@ubuntu:~# cat /proc/991/status | grep Cpus_allowed
Cpus_allowed:      3f
Cpus_allowed_list:      0-5
```

```
root@ubuntu:~# taskset -p 992
pid 992's current affinity mask: 3f
root@ubuntu:~# cat /proc/992/status | grep Cpus_allowed
Cpus_allowed:      3f
Cpus_allowed_list:      0-5
```

2.4.3 Run the test and while-loop on Guest VM

```
root@ubuntu:~# ./test
current cpu_mask: 3f
pin with cpu_mask: 05
current cpu_mask: 05
root@ubuntu:~# while true; do :; done &
[1] 675
```


2.4.4 Confirm the hypercall has been performed

```

Apr 06 16:24:28 ubuntu kernel: Filesystem uses xz compression. This is not supported
Apr 06 16:25:09 ubuntu kernel: kvm_read_affinity: vcpu->pid=991 affinity_mask=0-5 val=63
Apr 06 16:25:09 ubuntu kernel: kvm_read_affinity: vcpu->pid=992 affinity_mask=0-5 val=63
Apr 06 16:25:09 ubuntu kernel: kvm_assign_affinity: vcpu->pid=991, affinity_mask=0,2 val=5
Apr 06 16:25:09 ubuntu kernel: kvm_read_affinity: vcpu->pid=992, affinity_mask=0,2 val=5
Apr 06 16:25:09 ubuntu kernel: kvm_read_affinity: vcpu->pid=991 affinity_mask=0,2 val=5
Apr 06 16:25:09 ubuntu kernel: kvm_read_affinity: vcpu->pid=992 affinity_mask=0,2 val=5

```

2.4.5 Verify the affinity do apply on KVM host's CPUS

```

1  [|||||||||||||||||||||||||||||||||||||100.0%]  4  [||||| 0.0%]
2  [||||| 3.8%]  5  [||||| 1.3%]
3  [|||||||||||||||||||||||||||||||||||||100.0%]  6  [||||| 8.9%]
Mem[||||||||||||||||||||||||||||||||| 813M/1.93G] Tasks: 34, 57 thr; 3 running
Swp[||||| 0K/0K] Load average: 2.06 1.71 1.08
Uptime: 00:12:49

```

PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command
988	root	20	0	4246M	589M	14660	S	199.29.7	17:33.51	qemu-system-aarch64 -nographic -machine virt -m 512 -cpu	
991	root	20	0	4246M	589M	14660	R	100.29.7	8:35.60	qemu-system-aarch64 -nographic -machine virt -m 512 -cpu	
992	root	20	0	4246M	589M	14660	R	99.929.7	8:34.40	qemu-system-aarch64 -nographic -machine virt -m 512 -cpu	